Fine-Grained Computation Offload for Event-Driven Applications

Bojie Li
Pine AI

Abstract

Hardware accelerators now sit on the critical path of online serving: GPUs, TPUs, FPGAs, and increasingly *remote* services such as hardware security modules, post-quantum cryptography, and inference servers. While the accelerator works, the calling context stalls. For *fine-grained* offloads (microseconds to a few milliseconds), the classic responses both fall short: a context switch adds a wakeup latency comparable to the offload, and a busy-wait wastes the core. The principled fix is to *overlap* the offload with other useful work, but this requires injecting concurrency into the application. The real question is *how much the application must change*.

We study this across real, off-the-shelf servers and find a *spectrum*. At one extreme, an `LD_PRELOAD` M:N fiber runtime overlaps the offload with *zero* source modification, but works only for thread-per-connection servers and hits three architectural walls: synchronization below libc, thread identity in native thread-local storage, and per-request CPU that already exceeds the offload. At the other extreme, a *few lines* at the offload site route it through the application's own concurrency and work everywhere. Across ten servers spanning every concurrency model the change is **18–112 lines added and 0–1 modified**, and real overlap lands at 1.2–5.4× (17.3× for the transparent runtime), all measured on real hardware. A simple model predicts both the speedup and the code cost from two properties: the server's concurrency model and the offload's weight relative to per-request work, and the win is bounded at both ends, by the accelerator's throughput and by the server's own overlap capacity. A transparent page-protection conflict detector preserves correctness when the overlapped offload mutates shared state, validated on stock Redis.

Code: https://github.com/19PINE-AI/transparent-offload
Website: https://01.me/research/transparent-offload

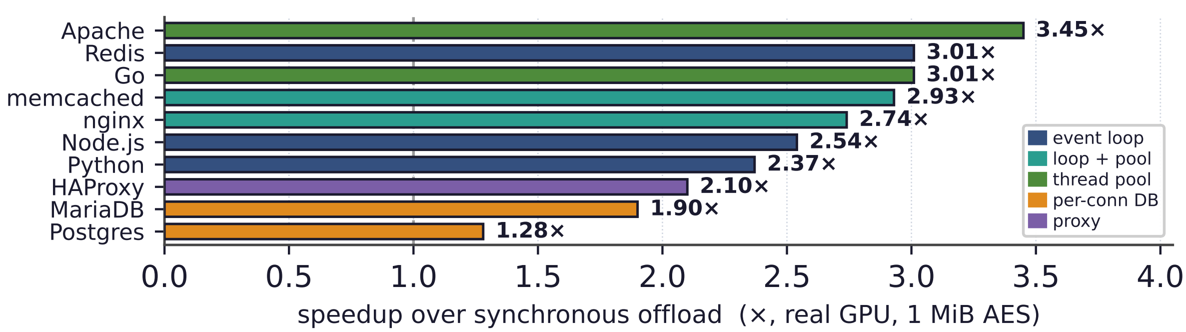

Figure 1. The headline result. A few lines of overlap recover 1.2–3.5× over a synchronous offload across ten servers (real GPU, 1 MiB AES; up to 5.4× at 8 MiB); the transparent runtime reaches 17.3× with zero edits. Color marks the concurrency model.

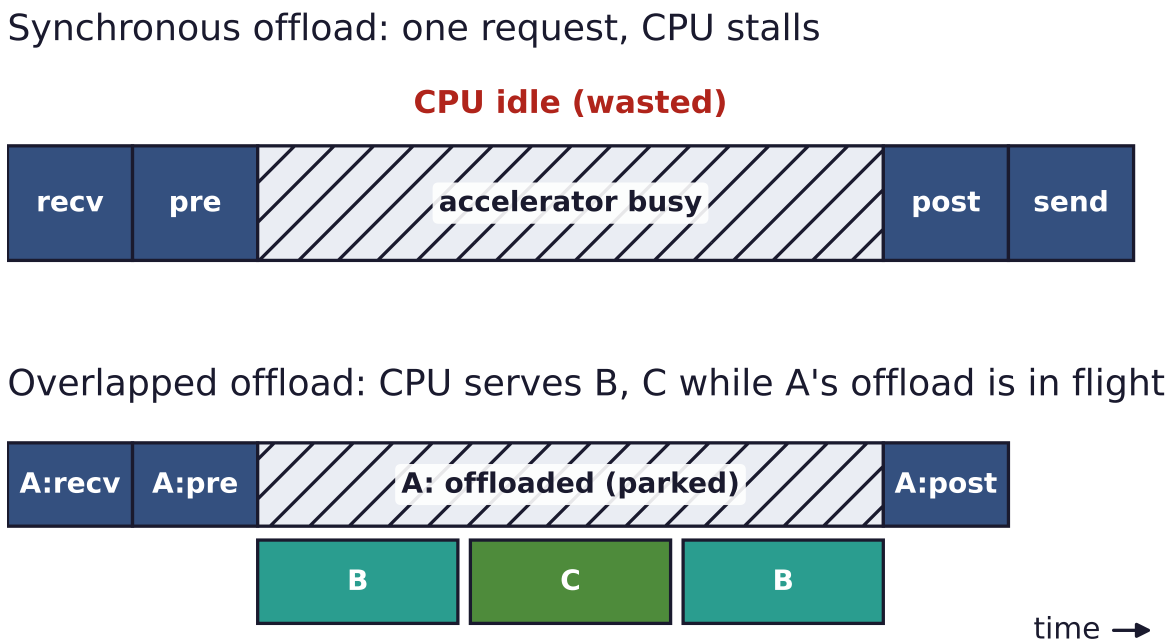

Figure 2. The problem. A serving request is *receive* → *pre-process* → *offload* → *post-process* → *send*. Top: with a synchronous offload, the CPU stalls while the accelerator works. For fine-grained offloads (microseconds to a few milliseconds), a context switch to fill the gap costs as much as the gap itself, and busy-waiting wastes the core. Bottom: *overlap* fills the gap with other requests' processing. The question of this paper is how much an existing server must change to do this.

1 Introduction

Hardware accelerators are now a standard part of online serving. GPUs, TPUs, and FPGAs accelerate inference, image processing, encryption, and compression, and a growing share of "acceleration" is in fact a *remote* call: to a hardware security module, a post-quantum key-encapsulation service [National Institute of Standards and Technology, 2024], or a co-located inference server [Crankshaw et al., 2017]. Across these cases the application has the same shape: receive a request, pre-process, invoke an intensive routine, post-process, respond. On an accelerator that routine becomes an *offload*: a submission to the device and a wait for its completion. This paper is about a deceptively simple question: *what should the CPU do while it waits for the accelerator?*

The textbook answers fail exactly in the regime that matters (Figure 2). *Blocking* lets the operating system run another thread, but a context switch costs several microseconds, comparable to a fine-grained offload itself, whether AES on a few kilobytes, a small inference, or an HSM round-trip. This is the "killer microsecond" regime [Barroso et al., 2017]: the switch wastes CPU and *adds* a wakeup delay to the request's tail latency. *Busy-waiting* hides the wakeup but burns a core doing nothing. The only principled answer is to *overlap* the offload with the processing of other requests, as high-performance systems already do by hand. Overlap, however, requires *concurrency* inside the application at the offload point, and existing servers express concurrency in many different ways. The central question of this paper is therefore not *whether* to overlap, but *how much an off-the-shelf server must be changed to do so.*

We answer this empirically, and the answer is a **spectrum** bounded by two extremes (Figure 9). At one extreme, overlap needs *no source modification at all*: a user-space M:N fiber runtime, loaded by `LD_PRELOAD`, turns each connection-handling thread into a lightweight fiber that yields at every blocking point while other fibers run on the same core. On a synchronous thread-per-connection handler it delivers 17.3× over a busy-wait synchronous baseline for GPU AES with the binary unchanged. But transparency reaches only so far, and mapping how far is itself a contribution. Stock binaries surface obstacles any threading-interposing tool must solve (notably a glibc condition-variable symbol-versioning hazard that silently corrupts some binaries), and beyond them lie *three architectural walls* that no engineering removes: storage engines that synchronize with raw `futex` system calls *below* libc, managed runtimes that keep thread identity in a native thread-local slot the loader cannot virtualize, and workloads whose per-request CPU already exceeds the offload. Crucially, the runtime engages only on *thread-per-connection* servers; an event-driven server has no per-connection thread to fiberize. So the class with the *most* to gain, where one synchronous offload stalls the *entire* server, is the class transparency cannot help.

At the other extreme, *a few lines* suffice. The recipe is uniform: replace the synchronous offload at its call site with an asynchronous one routed through the application's *own* concurrency mechanism, such as a module's thread pool, an event loop's deferred-response primitive, or a language runtime's tasks, and answer the request on completion. The edit is small because every modern server already contains machinery to suspend and resume work. We instantiate this recipe on **ten** off-the-shelf servers spanning every concurrency model: Redis, nginx, memcached, Node.js, Python/asyncio, Apache, Go, PostgreSQL, MariaDB, and HAProxy (seven stock server binaries; for Node.js, Python, and Go the server is an idiomatic handler on the stock runtime). In each case the change is **18 to 112 lines added, and zero or one existing line modified**. Measured on real hardware with no emulated latencies, the edit recovers 1.2–5.4× with no failed requests, bounded by the accelerator's throughput and by the concurrency the server can itself express (Figure 1, §7).

Behind these numbers is a simple predictive model, the paper's main intellectual contribution. Two properties of a server fix both the speedup *and* the code cost of overlapping an offload: its *concurrency model*, which predicts how much a minimal edit buys and how many lines it takes, and the offload's *weight relative to per-request CPU*, which predicts whether overlap helps at all. The model explains the full range of our measurements, including where overlap does *not* help, and tells an operator in advance which servers to touch and what to expect.

Overlap also introduces one correctness hazard: when post-processing mutates shared state, overlapping two requests can reorder their read-modify-write sequences and corrupt it. A transparent conflict detector, based on page protection and version snapshots, flags exactly these conflicts and serializes only the conflicting work, with no application annotation.

In summary, this paper makes the following contributions:

- **A predictive model** of fine-grained offload overlap in existing servers: speedup and code cost as a function of the server's concurrency model and the offload's weight relative to per-request work, validated across ten real servers (§5).
- **The transparent boundary, characterized**: a zero-modification M:N fiber runtime, the obstacle taxonomy it must overcome (including a reusable glibc condition-variable versioning bug), three architectural walls where transparency provably stops, and a classifier that predicts whether a binary is fiberizable (§3).

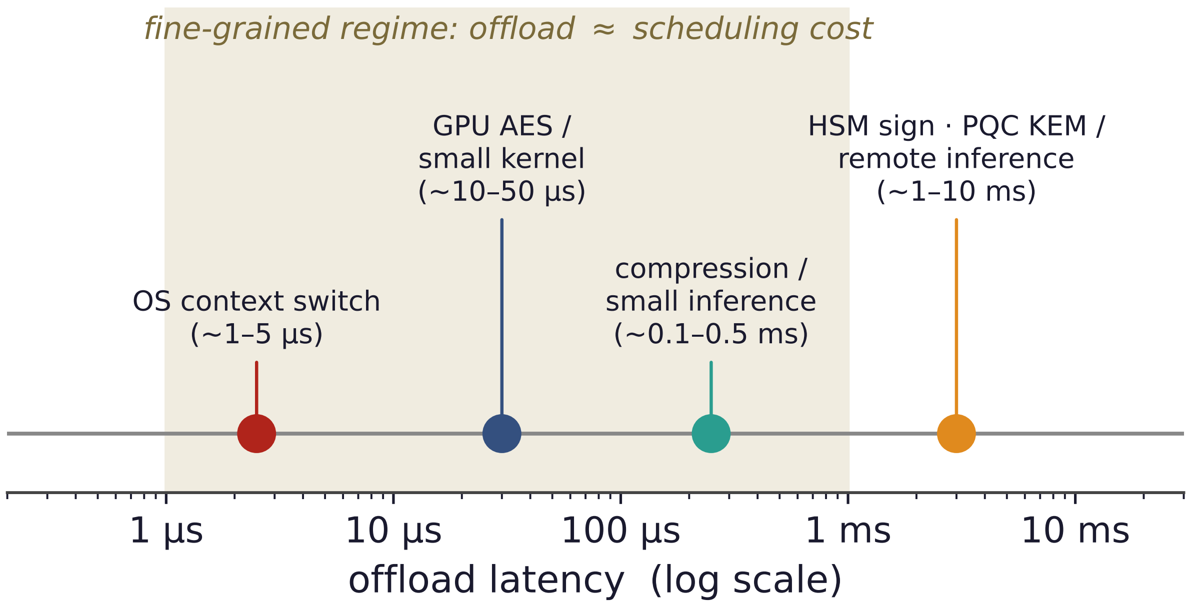

Figure 3. The fine-grained regime. Accelerator offloads span microseconds to milliseconds. In the shaded band, the offload latency is comparable to an OS context switch, so blocking pays a switch and a wakeup that rival the work itself, and busy-waiting wastes a core. This is exactly where overlapping the offload with other requests is the right answer.

- **A minimal-edit recipe and a cross-landscape study**: one offload-overlap pattern on ten off-the-shelf servers with 18–112 lines added and 0–1 lines modified, evaluated on real hardware (1.2–5.4×), with the win bounded at both ends by the accelerator's throughput and the server's own overlap capacity (§4, §7).
- **A transparent conflict detector** that keeps overlapped offload correct when it mutates shared state, with no application changes (§6).

2 Background and Motivation

Accelerators and the fine-grained regime. An accelerator turns a CPU-bound routine into a device operation: the CPU prepares an input buffer, submits it (a kernel launch, a DMA, or a network send), and later collects the result. These round-trips span microseconds to milliseconds (Figure 3), but all are *fine-grained* relative to operating-system scheduling: the offload finishes on the timescale of a context switch, so blocking and busy-waiting are both wasteful and overlapping the offload with other work is the only approach that neither wastes the CPU nor inflates latency.

How servers express concurrency. "Other work to overlap with" means *concurrent requests*, so a server's ability to overlap an offload is determined by how it runs concurrent requests in the first place. We group servers into five models that recur throughout the paper. A *single-event-loop* server (Redis, Node.js, a Python `asyncio` service) multiplexes all connections on one thread. An *event-loop-with-pool* server (nginx, memcached) runs a few such loops plus a worker pool. A *thread- or goroutine-pool* server (Apache, Go) dispatches each request to a pooled worker or runtime-scheduled task. A *proxy* (HAProxy) forwards each request through a native offload engine. A *process- or thread-per-connection* server (PostgreSQL, MariaDB) gives each connection its own backend. These models differ sharply in what a synchronous offload *costs* and in how overlap must be injected, differences this paper turns into a predictive model (§5).

The motivating catastrophe. The starkest case is a single event loop: because one thread handles every connection, a synchronous offload blocks *all* connections for its full duration, and throughput collapses to one request per offload latency while the hardware sits idle—a pathology of *structure*, not capacity. A few lines that move the offload off the loop recover more than an order of magnitude (§7). The event-driven servers that dominate modern serving thus have the most to gain from overlap and, as we show next, are also the ones a transparent runtime cannot help.

3 Transparent Overlap: The Zero-Edit Extreme

We begin at the zero-modification end of the spectrum: take an *unmodified* server binary and make it overlap its offloads, with no recompilation and no source access. The result is an existence proof that the mechanism works and, more importantly, a precise map of how far transparency can reach.

3.1 An `LD_PRELOAD` M:N fiber runtime

Our runtime (Figure 4) is a shared library loaded ahead of the application by `LD_PRELOAD` that interposes the standard threading and I/O symbols. When the application creates a connection-handling thread, the runtime instead spawns a *fiber*: a user-level thread with its own small stack and a register-only context switch of tens of nanoseconds, one to two orders of magnitude cheaper than a kernel switch [Li et al., 2023]. All fibers run on a single *carrier* OS thread. When a fiber would block, on socket I/O or on the offload, the runtime switches to another runnable fiber; a scheduler on the carrier waits for I/O readiness and offload completions and resumes the corresponding fiber. The application's handler keeps its plain synchronous shape and never learns that its "thread" is a fiber. Per-fiber state the C library treats as thread-local, such as `errno` and the OpenSSL error queue, is saved and restored at every switch.

On a synchronous thread-per-connection handler this works well. With a real GPU performing AES, the runtime overlaps 64 connections' offloads on one carrier core and reaches 17.3× the throughput of the same binary busy-waiting on each offload, with results verified bit-for-bit (11.9× over a baseline that blocks in the driver and lets the OS overlap the 64 threads); a DNN-inference server shows 11.8×. The rest of this section is about the mechanism's limits, which turn out to be the more useful contribution.

3.2 What it takes to run real binaries

Interposing the threading layer of a stock binary is harder than it sounds. The sharpest obstacle is a *symbol-versioning* hazard [Drepper, 2011]: the glibc condition-variable functions carry two incompatible ABIs under one name, so a naive interposer that defines an *unversioned* `pthread_cond_signal` breaks the linker's version-matched relocation and silently corrupts some binaries (MariaDB crashes at startup with a wild pointer) while sparing others (Figure 5).

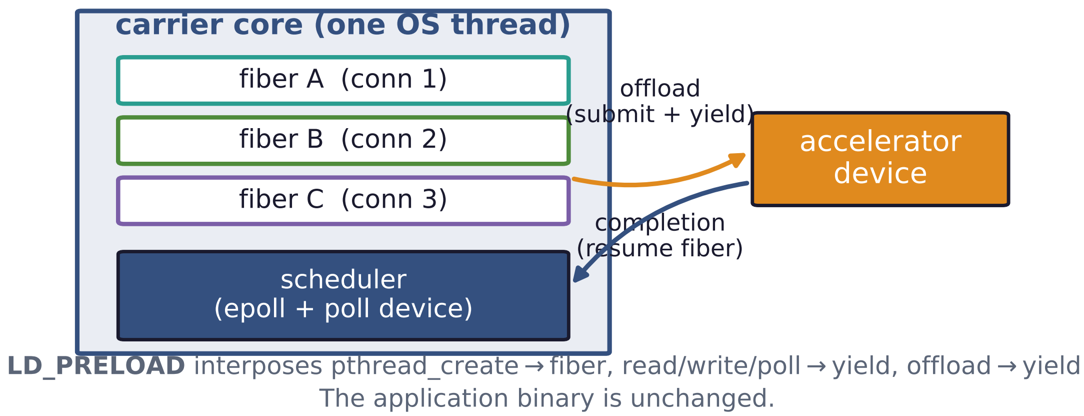

Figure 4. The transparent runtime. An `LD_PRELOAD` library turns each connection thread into a fiber on one carrier core. A fiber yields at its offload (and at socket I/O), and the scheduler runs another fiber until the device completes. The application binary is unchanged. On a synchronous thread-per-connection handler with a real GPU this overlaps 64 connections' offloads and delivers 17.3× over a busy-wait synchronous baseline (11.9× over OS-thread blocking).

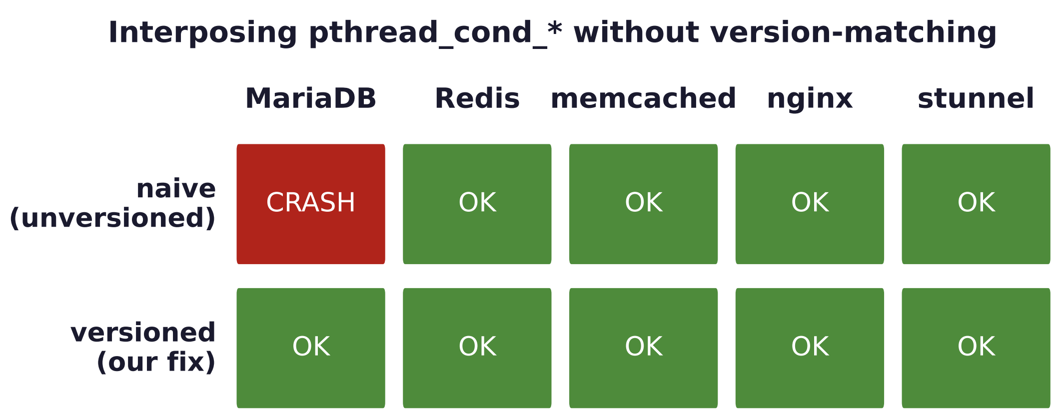

Figure 5. A reusable interposition hazard. The glibc condition-variable functions carry two incompatible ABIs under one name. A naive unversioned interposer breaks the version-matched relocation and corrupts *some* binaries (MariaDB crashes at startup) while silently sparing others. Version-matching the interposed symbols fixes all of them. Any threading-interposing tool must do this.

The fix is to export the interposed symbols at their exact version and resolve the real ones with `dlvsym`. The defect is latent in any threading-interposing tool, a reusable practitioner lesson. Several lesser obstacles—alternate I/O entry points, real-thread identity for libc's stack-bounds logic, carrier re-establishment after a daemon's `fork`, and priority inversion under real-time pinning—we resolved once each; Appendix B records them.

3.3 Three walls where transparency stops

Beyond engineering obstacles lie three *walls* that no amount of interposition removes (Figure 6). Each is a place where the application's behavior escapes the layer a preloaded library can intercept.

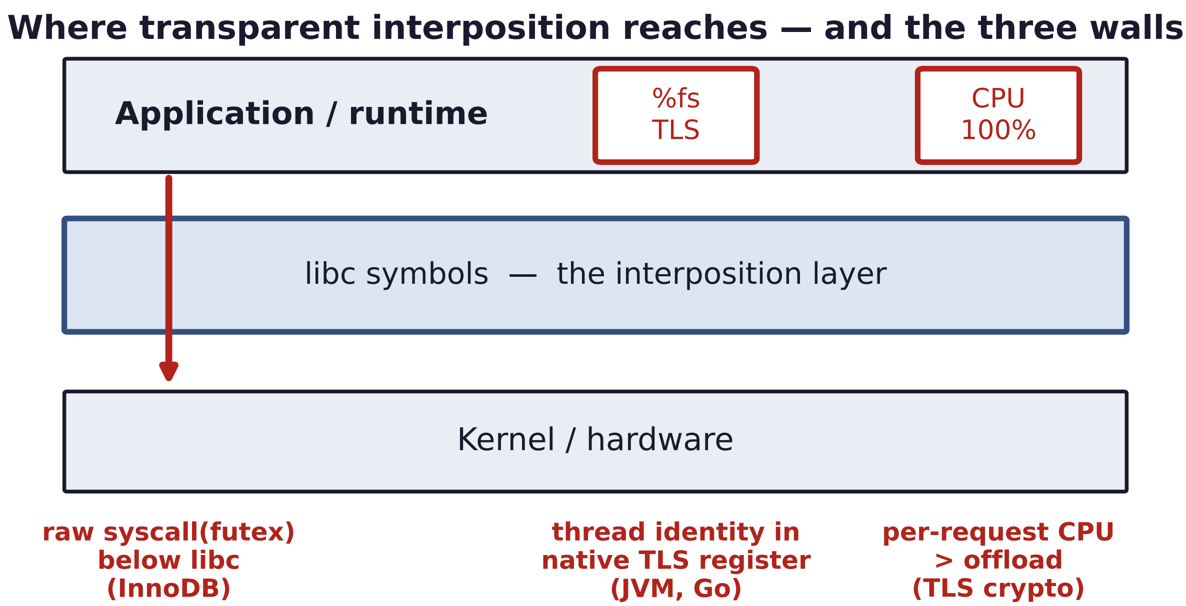

Figure 6. The three walls. Transparent interposition virtualizes the libc symbol layer but cannot reach synchronization issued as a *raw* system call below libc (InnoDB's `futex`), thread identity held in a native TLS register (the JVM, Go), or a workload whose per-request CPU already exceeds the offload. These bound where zero-edit overlap is possible.

Wall 1: synchronization below libc. A storage engine such as InnoDB does its internal locking with the `futex` system call [Franke et al., 2002] issued *directly*, bypassing the `pthread` layer. A userspace runtime interposes library *symbols*, not system calls, so a fiber that blocks in a raw `futex` stalls the entire carrier, and under contention the whole server deadlocks. We confirmed this with a live backtrace of the frozen carrier inside InnoDB's synchronization code.

Wall 2: thread identity in native TLS. A managed runtime such as the JVM stores "the current thread" in a native thread-local slot read directly from a CPU register, not through any library call. Symbol interposition cannot give fibers sharing one carrier distinct values of that slot, so after a fiber switch the runtime reads the *wrong* thread object and segfaults. We observe exactly this crash the moment two fiberized JVM requests interleave; the same mechanism applies to Go and .NET.

Wall 3: no idle CPU to reclaim. Overlap only helps if the CPU is idle during the offload. A thread-per-connection TLS terminator that spends real CPU on per-request crypto has no such idle time: the OS already overlaps its offloads across threads, while the single carrier serializes the crypto and pays an interposition tax on every yield, ending up 2–3$\times$ *slower* than native on the same core.

3.4 A classifier for fiberizability

The walls are predictable from a binary's behavior: we profile the *per-request blocking syscalls* a server makes under load. Event-driven servers do their I/O with `epoll` and `read` and have no per-connection thread to fiberize: the runtime loads safely but never engages. Thread-per-connection servers whose blocking is all at the libc layer have *near-zero* per-request `futex` traffic and are fiberizable. Engines that synchronize below libc have `futex` traffic that dominates their

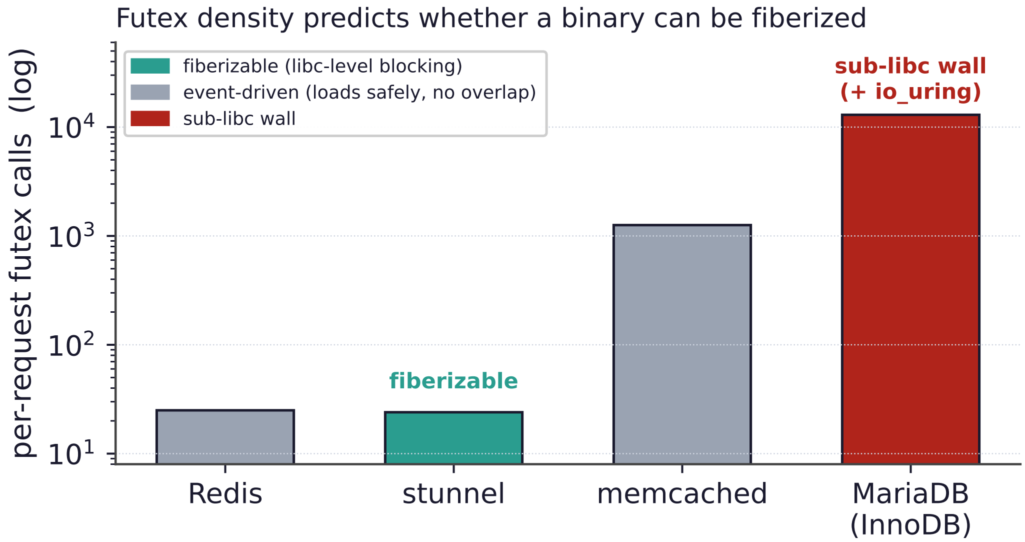

Figure 7. Predicting fiberizability. Per-request `futex` calls separate thread-per-connection servers that block at the libc layer (fiberizable: stunnel) from engines that synchronize below libc (InnoDB, ∼13,000, some 500× higher, plus `io_uring`), the wall of Figure 6. Event-driven servers (gray) have no per-connection thread to fiberize; their `epoll` profile flags them "loads safely, no overlap."

profile and use kernel-bypass I/O such as `io_uring` [Axboe, 2019]: that is the wall. InnoDB's per-request `futex` density is some 500× a fiberizable server's (Figure 7), a cheap check that tells an operator in advance whether a binary can be fiberized.

The verdict frames the rest of the paper. Transparency serves a narrow niche: thread-per-connection servers with light per-request CPU and a libc-level offload. By construction, it cannot help the event-driven servers that have the most to gain. For everyone else, a few lines suffice.

4 Minimal-Edit Overlap

If a server cannot be fiberized from the outside, it can almost always be overlapped by a small edit inside, because every modern server already contains machinery to *suspend* a request and *resume* it later—the machinery that lets it serve many connections at once. The recipe is uniform across servers:

1. at the offload call site, *submit* the work to a background executor instead of waiting.
2. *suspend* the current request using the server's own deferred-response primitive.
3. *resume* it and send the reply when the offload completes.

We distinguish *lines added* (new integration code) from *existing lines modified* (edits to code already in the server); the second is the invasive, maintenance-costly part, and zero is achievable wherever the server exposes a plugin or extension interface. Across ten servers spanning every concurrency model, the change is **18–112 lines added and 0–1 lines modified** (Figure 9).

The integration point follows the concurrency model. Servers with a *plugin or extension API* take the recipe with zero core edits: a Redis [Redis Ltd., 2024] module, an nginx [F5/NGINX, 2024] thread-pool add-on, an Apache [Apache Software Foundation, 2024] content handler,

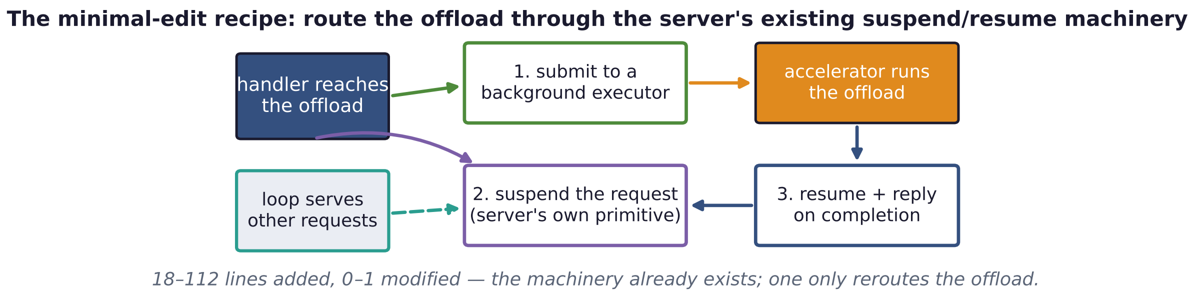

Figure 8. The minimal-edit recipe. At the offload call site, submit the work to a background executor, suspend the request with the server's own deferred-response primitive, and resume it to reply on completion. Meanwhile the server's loop serves other requests. The edit is small because the suspend/resume machinery already exists. One only reroutes the offload through it.

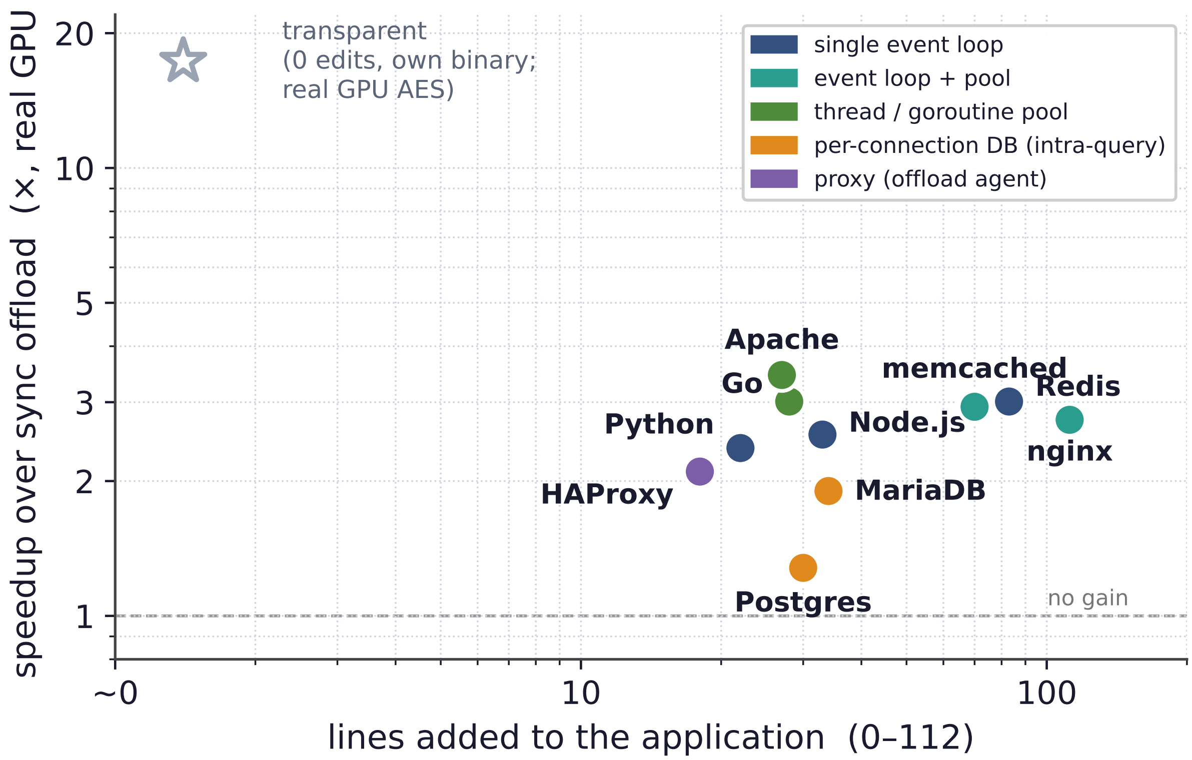

Figure 9. The spectrum (the paper in one plot): lines of integration code vs. speedup from overlapping a 1 MiB AES offload on a real, idle GPU. The few-line edit clusters at 2–3.5× (the offload is bandwidth-bound, so overlap fills the device but cannot exceed its throughput); the databases sit lower (intra-query pipelining only); the zero-line transparent runtime reaches 17.3× in its niche. Color encodes the concurrency model (§5).

PostgreSQL [PostgreSQL Global Development Group, 2024] and MariaDB [MariaDB Foundation, 2024] user-defined functions, a Node.js [OpenJS Foundation, 2024] N-API addon over libuv [libuv contributors, 2024], and a HAProxy [HAProxy Technologies, 2024a] deployment that streams each request to an external offload *agent* over its native SPOE engine [HAProxy Technologies, 2024b]. Servers backed by a *language runtime* need *no asynchronous code at all*: the goroutine scheduler overlaps a Go [The Go Authors, 2024] handler's plain blocking `cgo` offload, and a Python `asyncio` [Python Software Foundation, 2024] service offloads through

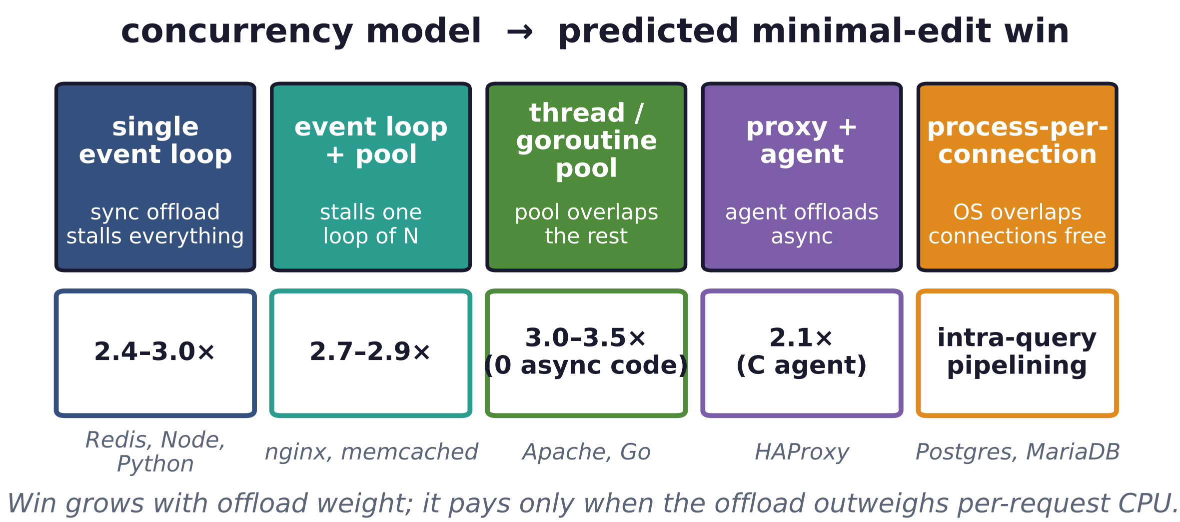

Figure 10. The predictive model, part one: the concurrency model determines what a synchronous offload costs and therefore how much a minimal edit recovers. The win ranges from *dramatic* (a blocked single loop) to *automatic* (a pool that overlaps for free) to *intra-query* (databases that already overlap across connections). Example servers are shown under each regime.

`run_in_executor` in one line. The lone case touching existing code is memcached [memcached, 2024], which has no asynchronous-command interface, so we add the park/resume transition to its state machine (70 lines, one modified)—the exception that proves the rule: the line count is dominated by whether the server already exposes a deferred-response primitive.

5 A Predictive Model of Offload Overlap

The measurements above follow a pattern. Two properties of a server predict both *how much* overlap helps and *how many lines* it takes: the server's *concurrency model* and the offload's *weight relative to per-request CPU work*.

Concurrency model predicts the regime. Figure 10 lays out the regimes, from *dramatic* to *none*. A *single event loop* is the extreme case—a synchronous offload stalls the whole server, so the few-line fix restores full concurrency and the win grows with offload weight up to the throughput ceiling—while an *event loop with a worker pool* stalls only one loop of several. A *thread or goroutine pool* overlaps offloads *automatically* with zero asynchronous code, and a *proxy* with a native offload engine makes async offload configuration-only. A *process- or thread-per-connection* server already overlaps *across* connections for free (the OS runs the backends concurrently), so its only edit-addressable win is *within* a single query that pipelines a serial loop of offloads.

Code cost follows the same axis. The number of lines is governed by whether the server exposes an asynchronous, deferred-response primitive. Plugin and extension interfaces (Redis, nginx, Apache, the databases) and language runtimes (Go, Python) provide one, so the integration is purely additive. A server without such a primitive forces the developer to add the suspend/resume transition to its request state machine by hand; memcached is our only

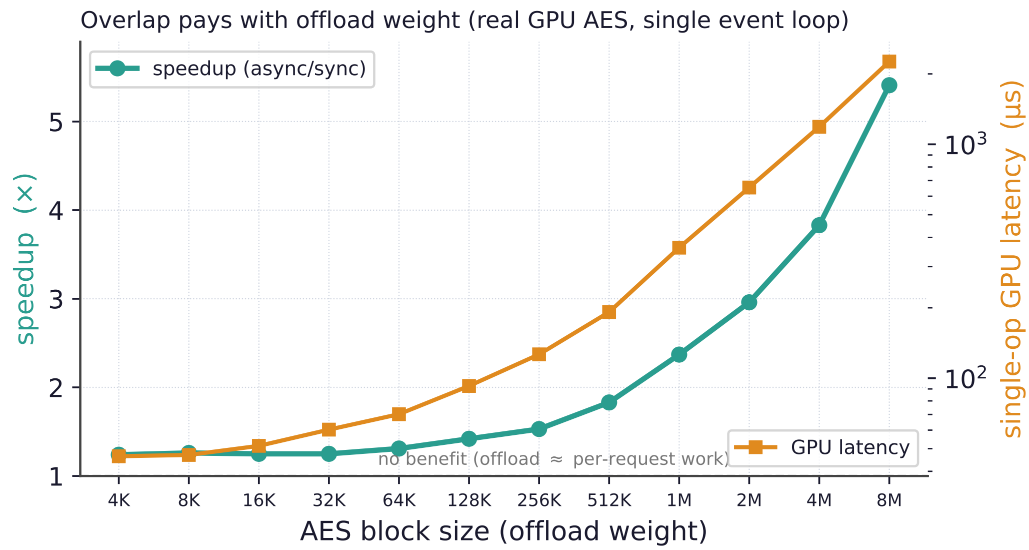

Figure 11. The predictive model, part two (measured, real GPU, idle): overlap pays only when the offload outweighs per-request CPU work. Sweeping a real GPU AES offload by block size (consecutive powers of two, 4 KiB–8 MiB) on a single-event-loop server, the same edit gives no benefit for a light block (1.24× at 4 KiB, 46 µs) and rises to 5.41× at 8 MiB (2.3 ms), approaching the GPU's bandwidth. Right axis: measured single-op GPU latency. This condition also explains the transparent runtime's third wall.

such case, and the only one that modifies an existing line.

Offload weight predicts whether overlap helps at all. The second axis is independent of the server. Overlap reclaims the wait for the offload, so if per-request CPU work is already comparable to the offload there is little to reclaim. A block-size sweep of a real GPU AES offload on a single event loop bears this out: a light block yields essentially no speedup and the win grows with weight toward the GPU's bandwidth (Figure 11). Overlap pays exactly when the offload outweighs the per-request CPU work *and* the accelerator is not already saturated—the same condition as Wall 3 of §3, seen from the transparent side.

Together the two axes form a map: given a server's concurrency model and an offload's latency, one can predict before writing any code whether to bother, how large the win will be, and roughly how many lines it will take.

6 Correctness: Overlapping Stateful Offloads

Overlap reorders work, and reordering is dangerous when the work touches shared state. If the post-processing after an offload does a read-modify-write on a global—a counter, a cache entry, a key in a store—overlapping two requests can interleave their sequences and lose updates. On a *stock Redis server* whose module command reads a key, runs a real GPU offload, and writes the incremented value back, unprotected overlap silently loses tens of thousands of updates (Figure 12, left). The more latency overlap hides, the wider the race it opens.

Two mechanisms restore correctness with no application change. *Per-connection ordering* is free: one fiber per connection serializes a connection's dependent requests. For state shared *across* connections, a transparent *conflict detector* write-protects the shared-state pages,

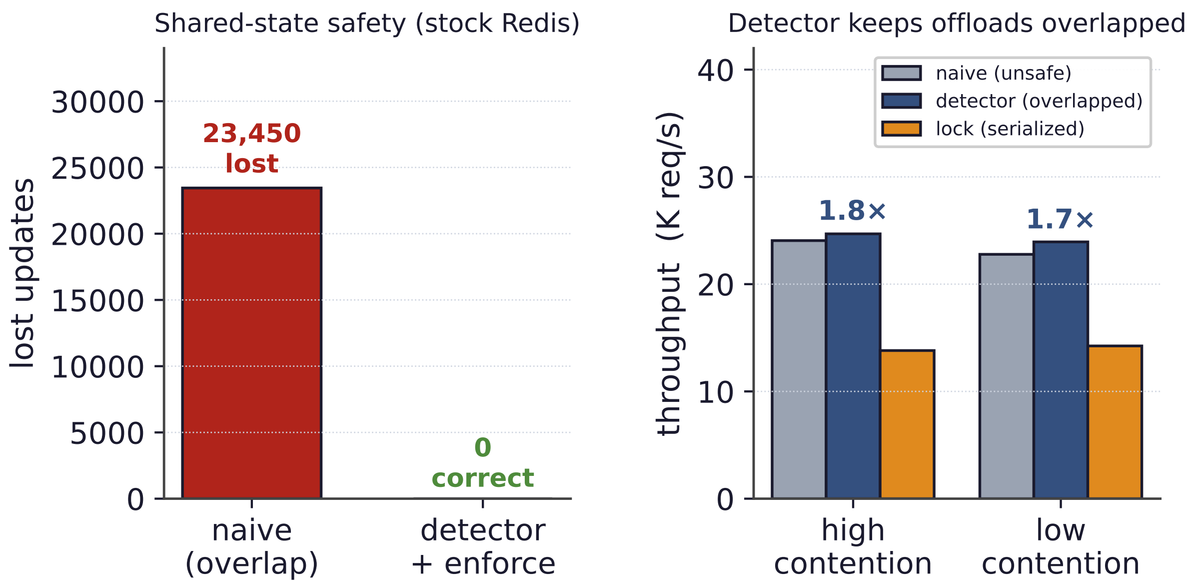

Figure 12. Keeping overlapped stateful offloads correct, measured on a stock Redis server with a real GPU offload. Left: unprotected overlap of a shared read-modify-write loses 23,450 updates. The page-protection detector flags 26,185 conflicts and, under enforcement, reduces lost updates to zero. Right: a coarse lock serializes the very offloads it protects (13.8K req/s), while the detector keeps non-conflicting offloads overlapped (24.7K req/s), 1.8× faster and within noise of unprotected overlap (24.1K req/s).

snapshots a version clock when a handler parks at its offload, and treats a write fault on a page modified during the park as a conflict; under enforcement it serializes only the conflicting handlers. On the stock-Redis workload it drives lost updates to *zero* while running 1.8× faster than a coarse lock that serializes the very offloads it protects; its cost over unprotected overlap is within measurement noise (24.7K vs 24.1K req/s) (Figure 12, right). The detector is a property of the overlap mechanism, not of any one integration, so it applies to both ends of the spectrum.

7 Evaluation

Setup. All cross-server offloads are real, on real hardware, with no emulated latencies; the one exception is the open-loop latency study of Figure 13, which uses a controlled emulated offload and is labeled as such. Experiments run on a server with an NVIDIA RTX PRO 6000 (Blackwell) GPU. The GPU path performs AES [OpenSSL Project, 2024] on its own CUDA stream, polled by `cudaEventQuery` [NVIDIA Corporation, 2024], with the block size sweeping from 4 KiB (launch-bound) to 8 MiB (bandwidth-bound). GPU AES is a stand-in for offloads that beat the host CPU (a modern core's AES-NI rivals a GPU on this cipher); we use it because it gives a cleanly tunable offload weight on real hardware. For the latency-bound *remote* class (HSM signatures, post-quantum KEMs, remote inference) we stand up a real TCP signer doing a genuine RSA-2048 signature per request. Event-driven servers are driven by standard load generators (`redis-benchmark`, `ab`). Figure 9 is the cross-server summary; the per-server code cost is in §4.

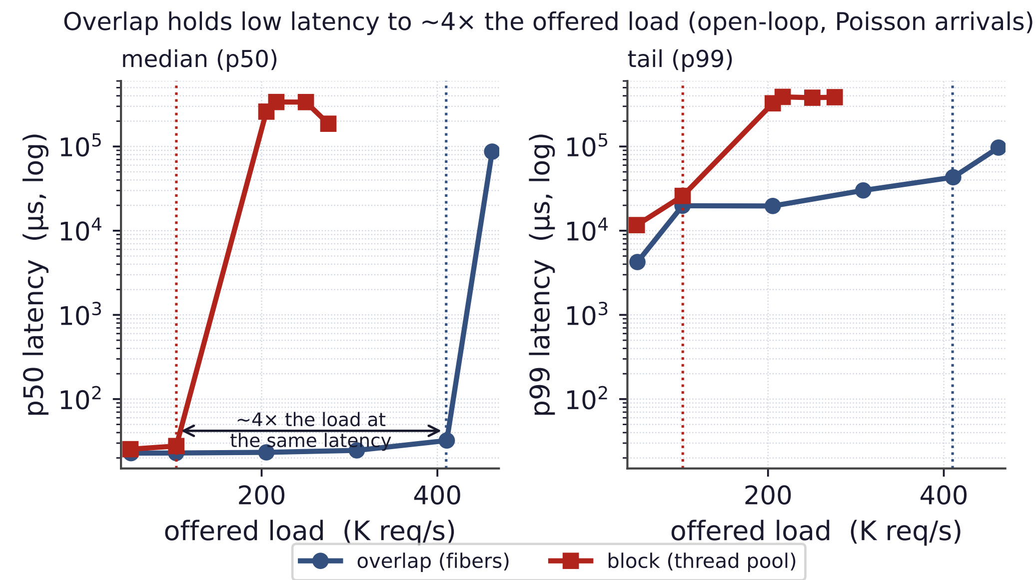

Figure 13. Latency under load (open-loop Poisson arrivals over a controlled 20 µs *emulated* offload, the one emulated experiment in the paper, used so offered load can be swept precisely; `runtime/openloop.csv`). Blocking on the offload drives both median (p50, left) and tail (p99, right) latency up at its saturation point near 103 K req/s, while overlapping holds low latency to roughly 4× that offered load (knee near 410 K req/s) before its own. Overlap is a latency win, not only a throughput win.

Event-driven servers and pools. On a 1 MiB GPU AES offload (realistic bulk crypto, idle GPU) the few-line edit recovers the expected win in every regime with no failed requests: single event loops and event-loop-with-pool servers cluster around 2.4–3×, and thread/goroutine pools (Apache, Go) reach 3.0–3.5× with *no* asynchronous code (Figure 1). The cluster sits at 2–3.5× because this offload is bandwidth-bound: overlap fills the device but cannot exceed its throughput. The proxy locates the bottleneck: HAProxy's win rides entirely on the offload *agent*, so a GIL-bound Python agent gains nothing while a C `pthread`-pool agent recovers 2.1× on the same offload.

Latency, not just throughput. Overlap is a latency win too. By keeping the CPU busy during offloads, the overlapping path holds both its median and tail (p99) latency low to roughly *four times* the offered load that blocking can sustain before its knee (Figure 13).

Offload weight and the two ceilings. Sweeping the GPU AES offload by block size on a single event loop confirms the weight dependence: no benefit for a light block, rising toward the GPU's bandwidth for a heavy one (Figure 11; full table in Appendix A). The ceiling is a property of the *device*: overlap recovers the utilization that blocking wastes but cannot exceed device throughput. A *latency-bound* offload lifts that ceiling because its device is not saturated. With our real RSA-2048 remote signer (821 µs round-trip) the same Python server reaches 3.53×—but not the order of magnitude a deep queue could in principle give, because the bottleneck moves to the server's own overlap capacity (the `asyncio`/GIL executor keeps only ∼6 requests truly in flight). Real overlap is thus bounded at *both* ends, by device throughput and by the concurrency the server can express; across our servers and offloads it lands at

1.2–5.4×.

Databases. A process- or thread-per-connection database already overlaps offloads *across* connections for free (the OS runs the backends concurrently), so the only edit-addressable win is *within* one query that pipelines many offloads. With a launch-bound GPU op this intra-query pipelining is modest (Figure 9), bounded by the device exactly as the model predicts.

8 Discussion and Limitations

The walls are fundamental, and the win is bounded. The three walls of §3 are properties of where an application places behavior, not gaps in our engineering: each sits below or around the symbol layer a preloaded library can touch, and they generalize to any storage engine with its own futexes, any managed runtime, and any CPU-bound handler—which is precisely why the minimal-edit path exists. The same idle-CPU condition (Wall 3 and Figure 11) also bounds when overlap pays at all: when per-request CPU rivals the offload there is little to reclaim, and the technique should not be applied.

Detector scope and complementarity. The conflict detector is a targeted safety net for the overlap hazard, not a general transactional memory; shared state outside the monitored segments, or correctness beyond last-writer-wins, would need stronger machinery. Finally, the two ends of the spectrum are a division of labor, not competitors: transparent fiberization serves thread-per-connection servers with no source access, and the minimal-edit recipe serves everything else—the event-driven bulk of modern serving best of all.

9 Related Work

Our work sits at an under-occupied point in the design space (Figure 14): low modification to *existing* servers *and* broad coverage across server types. Prior approaches achieve one or the other, but not both. Transparent threading libraries require writing the application in their style, point offload integrations target a single server, and async frameworks assume a rewrite.

Threads, events, and coroutines. Hiding I/O latency cheaply is an old debate, between an event-driven camp that structures the server as a state machine over an event loop at the cost of "stack ripping" [Adya et al., 2002] (Flash [Pai et al., 1999], SEDA [Welsh et al., 2001]) and a user-level-thread camp that keeps the synchronous style and yields at blocking points (Capriccio [von Behren et al., 2003], State Threads [State Threads Library, 2009], `libtask` [Cox, 2005], core-aware Arachne [Qin et al., 2018], and language-level threads such as goroutines [The Go Authors, 2024] and Java virtual threads [OpenJDK, 2023]). Our fiber runtime is in the second lineage with a fast register-only switch [Li et al., 2023], but the contribution is not another threading library: it is to *overlap an accelerator offload* in servers built either way, *quantify the modification cost* across real servers, and map where the transparent form provably stops.

Microsecond-scale systems. A body of work rebuilds the OS or runtime to run microsecond-scale tasks efficiently: dataplane operating systems such as IX [Belay et al., 2014], work-stealing schedulers such as ZygOS [Prekas et al., 2017], and core-reallocating runtimes such as

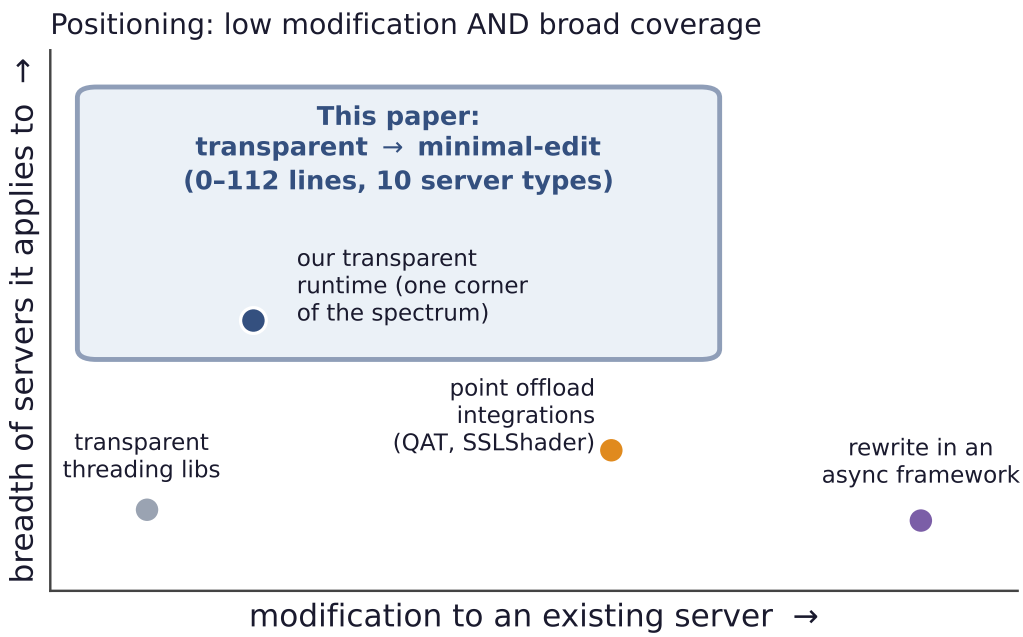

Figure 14. Positioning. Existing approaches to overlapping work are either low-effort but narrow (threading libraries, of which our own transparent runtime is one) or broad but high-effort (point integrations, async-framework rewrites). The transparent-to-minimal-edit spectrum reaches the desirable region: little modification to existing servers, across many server types.

Shenango [Ousterhout et al., 2019], Caladan [Fried et al., 2020], and Demikernel [Zhang et al., 2021], typically built over kernel-bypass I/O (DPDK [DPDK Project, 2024], `io_uring` [Axboe, 2019]). They attack the same killer microsecond problem [Barroso et al., 2017] but demand a new stack and rewritten applications. We instead ask how little it costs to overlap one offload inside existing, unmodified servers; the directions are complementary.

Full hardware offload. Another line moves the *entire* datapath onto hardware: KV-Direct [Li et al., 2017] runs a key-value store in the NIC, ClickNP [Li et al., 2016] compiles network functions to an FPGA, iPipe [Liu et al., 2019] offloads application logic onto SmartNICs (and must itself schedule the offloaded tasks' granularity, the dual of our problem), and GPUnet [Kim et al., 2014] lets GPU code drive the network. These eliminate the CPU's role and require rebuilding the application around the accelerator. Our target is the opposite and far more common case: the application stays on the CPU and must hide one fine-grained step's latency with almost no change.

Offload integrations and async frameworks. Closer to us, specific systems overlap specific offloads—TLS on GPUs (SSLShader [Jang et al., 2011]), QAT engines in nginx's async paths [Intel Corporation, 2024], and inference servers that batch GPU work (RedisAI [RedisAI, 2022], Clipper [Crankshaw et al., 2017])—but each is a point integration tuned to one server and offload. Async frameworks (libevent [Provos and Mathewson, 2024], libuv [libuv contributors, 2024], Seastar [ScyllaDB, 2024], `async/await`) and proxy offload engines (HAProxy SPOE [HAProxy Technologies, 2024b], Envoy `ext_proc` [Envoy Project, 2024]) make overlap the default, but only if the application is written in them from the start. We instead inject overlap into existing servers and give a *predictive model* of the win across the landscape.

Conflict detection. Dirty tracking via page protection, software distributed shared memory [Amza et al., 1996], and transactional memory [Herlihy and Moss, 1993] all detect or prevent conflicting concurrent writes. Our detector borrows the page-protection technique but is lightweight and specialized to the one hazard overlap introduces: a read-modify-write reordered across an offload.

Symbol interposition. The fragility of interposing versioned glibc symbols is folklore among practitioners. Our condition-variable finding documents a concrete, reproducible instance and its fix.

10 Conclusion

What should the CPU do while it waits for the accelerator? It should *overlap*: do other useful work while the offload is in flight. The real question is how much an existing server must change, and the answer is a spectrum from zero lines to about a hundred. At the zero-edit end a transparent runtime overlaps an unmodified thread-per-connection binary (17.3×), but it reaches a narrow niche, hits three architectural walls, and cannot help the event-driven servers with the most to gain. A few lines at the offload site, routed through a server's own concurrency mechanism, work everywhere, recovering 1.2–5.4× across ten off-the-shelf servers and runtimes on real hardware, bounded at both ends by the accelerator's throughput and the server's own overlap capacity. A simple model, concurrency model crossed with offload weight, predicts both the speedup and the code cost in advance, and a transparent conflict detector keeps overlap correct when it touches shared state. The result is a practical recipe, and a map, for hiding fine-grained accelerator-offload latency in the servers that run online services today.

Acknowledgements

This paper began as a draft written during the author's internship at Microsoft Research in 2017. Nine years later, with the help of Pine Copilot and Claude Code, the author finally brought it to completion. The work was produced using Pine Copilot's voice-directed *whisper coding* workflow, in which the author specifies, discusses, and reviews the work by voice while a coding agent—Claude Code with Claude Opus 4.8—carries out the planning, coding, experiments, and paper writing. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

A AES Block-Size Sweep (detailed values)

Table 1 gives the full per-size measurements behind the offload-weight curve of Figure 11, all on a real GPU on an *idle* device (no co-tenant compute), driving a single-event-loop server (Python `asyncio` with a 32-thread executor) at 50 concurrent clients. The AES block size is swept over consecutive powers of two from 4 KiB to 8 MiB. For each we report the measured single-op GPU latency (one offload in flight), the synchronous and asynchronous throughput, and their ratio. The speedup grows from 1.24× at 4 KiB (launch-bound: the offload is tens of microseconds, on the order of the server's own per-request work, so there is little to overlap) to 5.41× at 8 MiB (the offload is bandwidth-bound at ∼2.3 ms, so overlapping it across requests reclaims most of the wait), up to the GPU's throughput ceiling.

block size	GPU latency (µs)	sync (req/s)	async (req/s)	speedup
4 KiB	46.4	3750	4667	1.24×
8 KiB	47.0	3688	4650	1.26×
16 KiB	51.4	3657	4573	1.25×
32 KiB	60.3	3501	4371	1.25×
64 KiB	70.2	3326	4355	1.31×
128 KiB	92.7	2970	4218	1.42×
256 KiB	126.5	2706	4128	1.53×
512 KiB	191.8	2454	4494	1.83×
1 MiB	361.6	1538	3644	2.37×
2 MiB	653.6	966	2858	2.96×
4 MiB	1188.1	506	1937	3.83×
8 MiB	2258.6	227	1228	5.41×

Table 1. Real-GPU AES block-size sweep on a single-event-loop server (idle GPU). Source: `transparent-runtime/apps/aes_blocksize_py_results.csv`.

B Interposition Obstacles for the Transparent Runtime

Beyond the symbol-versioning hazard of §3 (Figure 5), running stock binaries under the `LD_PRELOAD` fiber runtime surfaced four lesser obstacles. Each was resolved once and is recorded here because any threading-interposing tool will meet them.

- **Alternate I/O entry points.** A server may do its socket I/O through `recv`/`send`/`poll` rather than `read`/`write`, so every blocking entry point the application can reach must be interposed to yield the fiber.
- **Real-thread identity.** `pthread_self` must return the genuine OS thread identity, because the C library's stack-bounds logic relies on it; returning a per-fiber value breaks libc internals.
- **Carrier re-establishment after `fork`.** A daemon that forks drops the carrier thread in the child, which must be re-established before any fiber can run.
- **Priority inversion.** Pinning the carrier to a real-time priority can invert priorities against a lock holder running at normal priority, stalling the carrier.

References

Atul Adya, Jon Howell, Marvin Theimer, William J. Bolosky, and John R. Douceur. Cooperative task management without manual stack management. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 2002.

Cristiana Amza, Alan L. Cox, Sandhya Dwarkadas, Pete Keleher, Honghui Lu, Ramakrishnan Rajamony, Weimin Yu, and Willy Zwaenepoel. TreadMarks: Shared memory computing on networks of workstations. *IEEE Computer*, 29(2):18–28, 1996.

Apache Software Foundation. Apache HTTP server. https://httpd.apache.org/, 2024.

Jens Axboe. Efficient IO with io_uring. https://kernel.dk/io_uring.pdf, 2019.

Luiz Barroso, Mike Marty, David Patterson, and Parthasarathy Ranganathan. Attack of the killer microseconds. *Communications of the ACM*, 60(4):48–54, 2017.

Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. IX: A protected dataplane operating system for high throughput and low latency. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 49–65, 2014.

Russ Cox. libtask: A coroutine library for C and Unix. https://swtch.com/libtask/, 2005.

Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 613–627, 2017.

DPDK Project. DPDK: Data plane development kit. https://www.dpdk.org/, 2024.

Ulrich Drepper. How to write shared libraries. https://www.akkadia.org/drepper/dsohowto.pdf, 2011.

Envoy Project. External processing filter (ext_proc). https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/ext_proc_filter, 2024.

F5/NGINX. nginx: Http and reverse proxy server. https://nginx.org/, 2024.

Hubertus Franke, Rusty Russell, and Matthew Kirkwood. Fuss, futexes and furwocks: Fast userlevel locking in Linux. In *Proceedings of the Ottawa Linux Symposium (OLS)*, pages 479–495, 2002.

Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 281–297, 2020.

HAProxy Technologies. HAProxy: The reliable, high-performance TCP/HTTP load balancer. https://www.haproxy.org/, 2024a.

HAProxy Technologies. Stream processing offload engine (SPOE) and protocol (SPOP). https://www.haproxy.com/documentation/, 2024b.

Maurice Herlihy and J. Eliot B. Moss. Transactional memory: Architectural support for lock-free data structures. In *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA)*, pages 289–300, 1993.

Intel Corporation. Intel quickassist technology (Intel QAT). https://www.intel.com/content/www/us/en/architecture-and-technology/intel-quick-assist-technology-overview.html, 2024.

Keon Jang, Sangjin Han, Seungyeop Han, Sue Moon, and KyoungSoo Park. SSLShader: Cheap SSL acceleration with commodity processors. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2011.

Sangman Kim, Seonggu Huh, Xinya Zhang, Yige Hu, Amir Wated, Emmett Witchel, and Mark Silberstein. GPUnet: Networking abstractions for GPU programs. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 201–216, 2014.

Bojie Li, Kun Tan, Layong (Larry) Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. ClickNP: Highly flexible and high performance network processing with reconfigurable hardware. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 1–14, 2016.

Bojie Li, Zhenyuan Ruan, Wencong Xiao, Yuanwei Lu, Yongqiang Xiong, Andrew Putnam, Enhong Chen, and Lintao Zhang. KV-Direct: High-performance in-memory key-value store with programmable NIC. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 137–152, 2017.

Bojie Li, Zihao Xiang, Xiaoliang Wang, Han Ruan, Jingbin Zhou, and Kun Tan. FastWake: Revisiting host network stack for interrupt-mode RDMA. In *Proceedings of the 7th Asia-Pacific Workshop on Networking (APNet)*, pages 1–7. ACM, 2023. doi: 10.1145/3600061.3600063.

libuv contributors. libuv: Cross-platform asynchronous I/O. https://libuv.org/, 2024.

Ming Liu, Tianyi Cui, Henry Schuh, Arvind Krishnamurthy, Simon Peter, and Karan Gupta. Offloading distributed applications onto SmartNICs using iPipe. In *Proceedings of the 2019 ACM SIGCOMM Conference*, pages 318–333, 2019.

MariaDB Foundation. MariaDB server: The open source relational database. https://mariadb.org/, 2024.

memcached. memcached: A distributed memory object caching system. https://memcached.org/, 2024.

National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS 203, NIST, 2024.

NVIDIA Corporation. CUDA C++ programming guide: Asynchronous concurrent execution. https://docs.nvidia.com/cuda/cuda-c-programming-guide/, 2024.

OpenJDK. JEP 444: Virtual threads (project Loom). https://openjdk.org/jeps/444, 2023.

OpenJS Foundation. Node.js: A JavaScript runtime built on V8. https://nodejs.org/, 2024.

OpenSSL Project. OpenSSL: Cryptography and SSL/TLS toolkit. https://www.openssl.org/, 2024.

Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 361–378, 2019.

Vivek S. Pai, Peter Druschel, and Willy Zwaenepoel. Flash: An efficient and portable web server. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 1999.

PostgreSQL Global Development Group. PostgreSQL: The world's most advanced open source relational database. https://www.postgresql.org/, 2024.

George Prekas, Marios Kogias, and Edouard Bugnion. ZygOS: Achieving low tail latency for microsecond-scale networked tasks. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 325–341, 2017.

Niels Provos and Nick Mathewson. libevent: An event notification library. https://libevent.org/, 2024.

Python Software Foundation. asyncio: Asynchronous I/O in CPython. https://docs.python.org/3/library/asyncio.html, 2024.

Henry Qin, Qian Li, Jacqueline Speiser, Peter Kraft, and John Ousterhout. Arachne: Core-aware thread management. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 145–160, 2018.

Redis Ltd. Redis: An in-memory data store. https://redis.io/, 2024.

RedisAI. RedisAI: A redis module for serving tensors and executing deep learning models. https://oss.redis.com/redisai/, 2022.

ScyllaDB. Seastar: A high-performance shared-nothing asynchronous C++ framework. https://seastar.io/, 2024.

State Threads Library. State threads library for internet applications. http://state-threads.sourceforge.net/, 2009.

The Go Authors. The Go programming language: Goroutines and the scheduler. https://go.dev/, 2024.

Rob von Behren, Jeremy Condit, Feng Zhou, George C. Necula, and Eric Brewer. Capriccio: Scalable threads for internet services. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, pages 268–281, 2003.

Matt Welsh, David Culler, and Eric Brewer. SEDA: An architecture for well-conditioned, scalable internet services. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, pages 230–243, 2001.

Irene Zhang, Amanda Raybuck, Pratyush Patel, Kirk Olynyk, Jacob Nelson, Omar S. Navarro Leija, Ashlie Martinez, Jing Liu, Anna Kornfeld Simpson, Sujay Jayakar, Pedro Henrique Penna, Max Demoulin, Piali Choudhury, and Anirudh Badam. The demikernel datapath OS architecture for microsecond-scale datacenter systems. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, pages 195–211, 2021.